

PRISM: Paradigm-Guided Reasoning with Iterative Solver Memory for Constraint Satisfaction

Anonymous submission

Abstract

Constraint Satisfaction Problems (CSPs) underpin numerous real-world applications including task planning, resource allocation, and scheduling. Large Language Models (LLMs) exhibit sharp accuracy degradation as CSP complexity grows; once the underlying solver encounters dozens of conflicts, performance approaches random and computational scaling techniques yield diminishing returns. We attribute this to two architectural limitations of existing neuro-symbolic and memory-augmented agent approaches: (1) repair mechanisms receive only coarse unsatisfiability signals and lack semantically grounded feedback, and (2) experience is either ignored across instances or stored as unverified natural-language prose without any mechanical quality control.

We propose **PRISM** (Paradigm-guided Reasoning with Iterative Solver Memory), a neuro-symbolic framework built around a single observation: in CSP solving, every reasoning step admits mechanical certification by an SMT solver, and this enables an end-to-end SMT-grounded feedback loop at two granularities. (i) At the *paradigm level* (offline), candidate solving patterns distilled from successful trajectories are screened by Z3 along three axes—soundness via sampling, effect via non-contradiction and non-vacuousness checks ($\phi_{\text{pre}} \wedge \neg \text{op}$ must be SAT), and trigger precision via structured kind-set selectivity—before admission to the paradigm library. (ii) At the *repair-trajectory level* (online), a typed memory records canonicalised UNSAT cores and structured repair signatures; UNSAT-core stagnation and structured loop detection trigger a four-level adaptive escalation protocol (switch target $\rightarrow$ switch type $\rightarrow$ revert from a bounded checkpoint stack $\rightarrow$ full retranslate) with a worst-case bounded LLM-call budget per puzzle.

To our knowledge PRISM is the first framework to apply SMT-based mechanical screening to LLM-induced reasoning rules at the paradigm level and to combine it with SMT-grounded escalation at the repair level. We evaluate PRISM on the ZebraLogic benchmark across six complexity scales (3×5 to 6×6) against seven neuro-symbolic and memory-based baselines, and we formulate a *paradigm transfer rate* protocol to assess cross-scale and cross-problem generalisation of accumulated experience.

Introduction

Constraint Satisfaction Problems (CSPs) form the backbone of numerous real-world applications including task planning, resource allocation, and combinatorial optimization.

Formally, a CSP is defined as a tuple $\mathcal{P} = (\mathbf{X}, \mathbf{D}, \mathbf{C})$ where $\mathbf{X}$ is a set of variables, $\mathbf{D}$ specifies finite domains for each variable, and $\mathbf{C}$ is a set of constraints. A solution is a complete assignment of variables that simultaneously satisfies all constraints.

Recent advances in Large Language Models have demonstrated impressive capabilities across diverse reasoning tasks. However, LLMs struggle with structured constraint reasoning—a phenomenon systematically documented in recent benchmarks. The ZebraLogic benchmark (Lin et al., 2025) reveals a critical limitation: as puzzle complexity scales from 3×5 to 6×6 logical grids, all evaluated models exhibit sharp accuracy degradation. When the Z3 constraint solver encounters more than 20 conflicts, model performance approaches random baselines, and computational scaling techniques (Best-of-N sampling, self-verification) yield diminishing returns.

This performance cliff is not merely a scale-dependent phenomenon but stems from a fundamental architectural limitation: LLMs lack systematic strategies for navigating complex constraint spaces. Existing neural-symbolic approaches (Pan et al., 2023; Olausson et al., 2023) adopt a three-stage translate-execute-repair pipeline where LLMs convert natural language constraints to formal representations, solvers perform deduction, and LLMs attempt repairs upon failure. This paradigm has achieved improvements over pure language model reasoning but exhibits two critical deficiencies:

(1) Repair Inefficiency: When solvers return UNSAT, repair mechanisms receive only coarse feedback (e.g., raw error strings or generic unsatisfiability statements) lacking semantic information about constraint conflicts. LLMs thus iteratively generate near-identical erroneous repairs—a phenomenon documented as “stagnation” in recent work. Critically, the system cannot distinguish whether contradictions arise from “unsolvable formulations” or “over-constrained translations,” nor can it detect repair cycles.

(2) Absence of Experience Accumulation: Each puzzle is solved independently; successful translation patterns, error diagnoses, and repair strategies accumulated on one problem are completely discarded when switching to the next. Identical constraint translation errors recur across different problem instances, necessitating redundant repair iterations.

Agent memory methods (Shinn et al., 2023; Zhao et al., 2024; Ouyang et al., 2025) explore cross-task experience accumulation through trajectory comparison and heuristic extraction. However, existing memory frameworks face inherent limitations when applied to CSP reasoning: (a) memory units are unverified natural language prose lacking formal correctness guarantees, and (b) task-level memory granularity mismatches the operation-level semantic patterns required for constraint reasoning.

Key Insight: CSP solving exhibits a property absent in general agent tasks: every reasoning step admits mechanical certification by an SMT solver. This property reshapes how experience can be accumulated. Paradigms can be subjected to an automated quality-screening protocol before storage—rather than relying on subjective LLM self-evaluation—even though the screening itself is sample-based and yields probabilistic, not absolute, guarantees. Repair failures additionally produce UNSAT cores, minimal unsatisfiable constraint subsets that localize contradictions with semantic precision.

We introduce **PRISM** (Paradigm-guided Reasoning with Iterative Solver Memory), a memory-augmented neuro-symbolic framework designed specifically for CSP solving. PRISM operates via two decoupled phases:

Offline Phase (one-time execution): Given a corpus of training puzzles, the system:

1. Collects successful solving trajectories via LLM+Z3 interaction
2. Identifies Key Decision Points (KDPs)—steps where domain reductions are significant or inference types are non-trivial
3. Clusters KDPs via cross-trajectory similarity on constraint-type feature vectors
4. Abstracts reusable solving paradigms via LLM synthesis
5. Filters candidates through triple Z3 verification (soundness, effect, trigger precision)
6. Persists verified paradigms in a Paradigm Library

Online Phase (per-puzzle execution): For each target puzzle, PRISM:

1. Extracts constraint-type signatures from current solver states
2. Retrieves relevant paradigms via two-layer filtering (type matching + semantic verification)
3. Injects paradigms as structured hints to guide LLM inference
4. Performs real-time Z3 verification of each reasoning step
5. Upon UNSAT outcomes, extracts UNSAT cores for causal analysis
6. Records typed repair attempts in a Repair Trajectory Memory
7. Detects stagnation (Jaccard similarity of UNSAT cores) and loops (cosine similarity of repair actions)
8. Triggers four-level strategy escalation (L1: switch repair target; L2: switch repair type; L3: revert to recent SAT checkpoint; L4: full retranslation)

9. Stages successful repair patterns to a candidate pool $\mathcal{L}^\dagger$, which is batch re-verified (via the same triple-verification protocol) before promotion to the live paradigm library $\mathcal{L}$

Main Contributions. We organize our contributions around two primary mechanisms—paradigm-level certification and repair-level adaptive escalation—supported by an enabling pipeline and a dedicated evaluation methodology.

- **C1 (Primary) — SMT-Screened Paradigm Representation.** We propose a paradigm representation in which trigger conditions, inference operations, and outcomes carry Z3 pre/post-conditions, allowing each candidate paradigm to pass an empirical three-axis screening protocol (soundness sampling, effect via non-contradiction + non-vacuousness, trigger selectivity) before library admission. To our knowledge this is the first LLM-memory framework that applies SMT-based automated screening at the paradigm level, providing a quality filter absent in prior natural-language memory approaches.
- **C2 (Primary) — Typed Repair Trajectory Memory with Four-Level Adaptive Escalation.** We design a structured memory that models constraint repair as a sequence of typed records (UNSAT core, error class, structured action signature, outcome). Coupled with a canonicalized UNSAT-core stagnation detector and a structured loop detector, the memory triggers a four-level escalation protocol (switch target $\rightarrow$ switch type $\rightarrow$ revert checkpoint $\rightarrow$ full retranslate) that mechanically breaks repair stagnation and guarantees bounded online cost.
- **C3 (Enabling Pipeline).** We instantiate C1 with a fully unsupervised offline pipeline that operationalizes paradigm distillation: trajectory collection, Key Decision Point (KDP) identification, cross-trajectory clustering, LLM abstraction, and triple verification. This pipeline produces the verified library that the online stage relies on, and is replaceable in principle by any other distillation procedure that respects the same SMT-screening interface.
- **Evaluation Contribution — Paradigm Transferability Protocol.** We formulate and report *paradigm transfer rate* along two axes—cross-scale (L1) within a benchmark and cross-problem-type (L2) across benchmarks—as an evaluation dimension for memory-augmented neuro-symbolic reasoning. This shifts the evaluation focus from per-puzzle accuracy alone to whether accumulated experience generalizes.

Experimental Results: Evaluation on ZebraLogic (6 complexity scales, 3×5 to 6×6) demonstrates consistent improvements over seven baselines including pure CoT, logic-specific methods (Logic-LM, ExpeL), and neural-symbolic baselines. The largest gains appear on the most difficult 6×6 puzzles, validating our hypothesis that experience accumulation provides exponentially increasing value for harder problems.

The remainder of this paper is organized as follows. Section 2 reviews related work on neural-symbolic reasoning and agent memory. Section 3 presents the PRISM methodology in detail. Section 4 describes experimental setup and

baselines. Section 5 reports results and analysis. Section 6 concludes with implications and future directions.

Related Work

Neural-Symbolic Constraint Reasoning

Early neural-symbolic approaches (Mao et al., 2019; Yi et al., 2019) combined perception networks with symbolic programs. Recent work focuses on LLM-based constraint reasoning. Pan et al. (2023) propose Logic-LM, combining language models with symbolic executors for mathematical reasoning. Their translate-execute-repair pipeline forms the foundation for subsequent neuro-symbolic CSP work. Olausson et al. (2023) employ Z3 for explicit constraint verification, while Ye et al. (2023) integrate formal methods into language model reasoning pipelines.

However, existing approaches exhibit two critical limitations: (1) repair mechanisms receive only coarse error signals, leading to stagnation; (2) each problem is solved independently without cross-problem experience transfer. PRISM addresses both through formally-grounded paradigm libraries and typed repair memory.

Agent Memory and Experience Accumulation

Memory-augmented agents (Shinn et al., 2023; Zhao et al., 2024; Ouyang et al., 2025) accumulate experiences across tasks through trajectory comparison and heuristic extraction. These methods demonstrate effective cross-task transfer in web navigation, embodied tasks, and software engineering.

Critically, existing memory frameworks have two limitations when applied to CSP reasoning: (a) memory units are unverified natural-language heuristics, and (b) task-level granularity mismatches the operation-level semantic patterns required. PRISM overcomes these through Z3-verified paradigms and operation-level repair records.

Formal Methods and Constraint Solving

Classical CSP solvers (Dechter, 2003; Russell & Norvig, 2021) employ systematic search and constraint propagation. SMT solvers like Z3 (de Moura & Bjørner, 2008) provide mechanical verification of logical formulas. Recent work increasingly leverages formal methods: Talmor et al. (2019) use automated theorem provers for reasoning, while Sinha et al. (2021) combine neural networks with formal verification.

PRISM’s distinctive use of Z3 is as a *quality-screening oracle* for candidate experience, not merely as a back-end solver. The oracle role enables empirical certification of experience without human annotation, complementing—rather than replacing—the solver role exploited by prior neuro-symbolic CSP work.

Learning Reusable Inference Rules

PRISM is closest in spirit to a long line of work on *learning reusable inference rules*. Explanation-Based Learning (EBL) generalizes successful problem-solving traces into reusable schemas with provable correctness derived from a domain theory (Mitchell, Keller, & Kedar-Cabelli, 1986; DeJong & Mooney, 1986). Macro-operator learning

compresses sequences of primitive operators into reusable macros (Korf, 1985), and Minton (1990) studies the utility tradeoff between macro coverage and matching cost. In the symbolic-solver tradition, conflict-driven clause learning (Marques-Silva & Sakallah, 1996) and lemma generation in SMT/model-checking (Bradley, 2011) accumulate experience from *failed* search branches and feed it back as new constraints. Minimal unsatisfiable subset enumeration (Liffiton & Sakallah, 2008) provides the algorithmic underpinning that PRISM’s UNSAT-core analysis builds on.

PRISM occupies a specific position in this spectrum. Compared to EBL macros, our paradigms are induced from LLM-generated traces (which are *noisy* rather than provably sound) and are certified by Z3 *empirically* via sampling-based screening rather than by first-principles proof from a hand-built domain theory. Compared to clause / lemma learning in classical SMT, PRISM operates at the level of *reusable inference patterns across problem instances* rather than learning conflict clauses scoped to a single solver invocation. To our knowledge, PRISM is the first to combine LLM-induced inference rules with SMT-based empirical screening, occupying the gap between provably-sound EBL macros (limited by a hand-built domain theory) and unverified LLM-memory heuristics (limited by lack of mechanical quality control).

Methodology

Problem Definition and Notation

A Constraint Satisfaction Problem is formally defined as $\mathcal{P} = (\mathbf{X}, \mathbf{D}, \mathbf{C})$ where:

- $\mathbf{X} = \{x_1, x_2, \dots, x_n\}$ is a set of variables
- $\mathbf{D} = \{D_1, D_2, \dots, D_n\}$ specifies finite domains ($x_i \in D_i$)
- $\mathbf{C} = \{c_1, c_2, \dots, c_m\}$ is a set of constraints

A solution is a complete assignment $\theta : \mathbf{X} \rightarrow \bigcup_i D_i$ such that $\theta(x_i) \in D_i$ and $\forall c_j \in \mathbf{C} : c_j(\theta)$ holds.

Solving Trajectory: For puzzle instance $\mathcal{P}$, a solving trajectory is a temporal sequence of reasoning states:

$$\mathcal{T} = \langle (S_0, a_1, S_1), (S_1, a_2, S_2), \dots, (S_{T-1}, a_T, S_T) \rangle$$

where $S_t = (C_t, \delta_t)$ is the solver state at step t , with C_t being formalized constraints and δ_t being domain assignments. Action a_t is LLM-generated inference (natural language statement + Z3 formalization). Trajectory $\mathcal{T}$ is *successful* if S_T satisfies all constraints with Z3 returning SAT and assignment matching ground truth.

Solving Paradigm: We define a solving paradigm as a five-tuple:

$$P = (\mathcal{Q}, \mathcal{O}, \phi_{\text{pre}}, \phi_{\text{post}}, \mathcal{S})$$

where:

- $\mathcal{Q}$ specifies trigger conditions (constraint types and solver state features)
- $\mathcal{O}$ is the inference operation (parameterized template, e.g., $\text{pos}(X) = k + d$)
- ϕ_{pre} is the Z3-formalized precondition assertion
- ϕ_{post} is the Z3-formalized postcondition assertion

- $\mathcal{S}$ documents applicable scope (puzzle types and scales)

ϕ_{pre} and ϕ_{post} are written in SMT-LIB form so they can be mechanically checked by Z3 at each application—giving the paradigm a structured, executable interface that prior natural-language memory formats lack. We do not claim absolute formal correctness: the screening protocol described in Section 3.3.4 is sample-based and provides probabilistic acceptance guarantees rather than first-principles proofs.

Paradigm: Position in the Inference-Rule Spectrum To prevent confusion with adjacent notions, we briefly locate PRISM paradigms in the spectrum of learned inference rules:

- **Lemma / learned clause (SAT/SMT):** Ground-truth, proof-derived from a single solver invocation, scoped to that invocation; correctness is automatic.
- **Macro-operator (EBL):** Generalised from one or more successful traces under a hand-crafted domain theory; provably sound by construction; scope is the domain theory’s expressive range.
- **Unverified prior LLM-memory unit:** Natural-language heuristic mined from agent trajectories; cross-task reusable; quality uncertified — relies on subjective LLM self-evaluation.
- **PRISM paradigm (this work):** LLM-induced from clustered successful trajectories, then *empirically* screened via Z3 sampling along three axes. It is cross-instance reusable (unlike a lemma), has no hand-built domain theory (unlike EBL), and carries an SMT-executable interface and a non-trivial empirical quality certificate (unlike prior LLM-memory units).

PRISM paradigms thus sit between provably-sound EBL macros (limited by the cost of a formal domain theory) and unverified LLM heuristics (limited by lack of mechanical quality control), trading a probabilistic acceptance guarantee for substantially wider applicability.

PRISM Framework Overview

PRISM operates via two decoupled stages: offline distillation and online inference.

Offline Phase: Given training puzzles $\{\mathcal{P}_1, \dots, \mathcal{P}_N\}$:

1. Collect successful solving trajectories via LLM+Z3
2. Identify Key Decision Points (domain reductions ≥ 2 or non-trivial step types)
3. Cluster KDPs via complete-linkage agglomerative clustering on constraint-type features
4. Abstract paradigms via LLM synthesis from cluster representatives
5. Filter candidates through triple Z3 verification (soundness ≥ 0.90 , effect ≥ 0.80 , precision ≥ 0.80)
6. Persist verified paradigms in Paradigm Library $\mathcal{L}$

Online Phase: For each puzzle $\mathcal{P}^*$:

1. Extract constraint-type signature $\sigma(S_t) = \text{sorted}(\{type(c) \mid c \in C_t\})$

2. Retrieve paradigms via Layer-1 (type matching) + Layer-2 (LLM semantic judgment)
3. Execute Z3 consistency pre-check; inject passing paradigms as hints
4. Generate and verify LLM inference steps via Z3
5. Upon UNSAT, extract UNSAT core and perform causal attribution
6. Record typed repairs in Repair Trajectory Memory $\mathcal{M}$
7. Detect stagnation (Jaccard similarity) and loops (cosine similarity); trigger strategy escalation
8. Write successful repairs back to augment $\mathcal{L}$

Offline Stage: Paradigm Distillation

Trajectory Collection Given $N = 600$ training puzzles across difficulty levels, we execute LLM+Z3 solving $r = 3$ times per puzzle with temperature = 0.7 for inference diversity. We retain only successful trajectories where Z3 returns SAT with assignments matching ground truth. This yields approximately 1500 successful trajectories as the distillation source.

Each trajectory step records: Z3 verification labels (SAT/UNSAT/ERROR), domain change records Δ_t (variables with reduced domains), constraint type signatures, and preliminary step classification.

Key Decision Point Identification A step (S_{t-1}, a_t, S_t) is a Key Decision Point if any of the following holds:

- **Condition A (Significant Domain Reduction):** $\exists x : |\delta_{t-1}(x)| - |\delta_t(x)| \geq 2$ — at least one variable’s domain shrinks decisively in a single step.
- **Condition B (Non-Trivial Step Type):** step classification is CHAIN_PROPAGATION or CONTRADICTION_ELIMINATION.
- **Condition C (Aggregate Information Gain, soft):** the total log-ratio of domain reductions across all variables exceeds a bit-threshold τ_{ig} :

$$G(t) = \sum_{x: |\delta_t(x)| < |\delta_{t-1}(x)|} \log_2 \frac{|\delta_{t-1}(x)|}{|\delta_t(x)|} \geq \tau_{\text{ig}} \quad (\text{default } 1.0 \text{ bit}).$$

Condition C complements the hard per-variable cutoff of A: it captures *gradual tightening* steps that move several domains from $5 \rightarrow 3$ or $4 \rightarrow 2$ — individually below the threshold of A but collectively constituting a meaningful inference. Empirically the dominant useful pattern in harder puzzles (e.g. 6×6 ZebraLogic) is precisely such gradual tightening, so condition C broadens paradigm coverage where the library most needs it. Step types are assigned via lightweight rule-based classification on natural-language action text.

Cross-Trajectory Clustering and Paradigm Abstraction
Feature Representation: Each KDP is represented as:

$$\mathbf{f}(\text{kdp}) = [\mathbf{h}_{\text{ct}}, \mathbf{b}_{\text{dom}}, \mathbf{e}_{\tau}, n_{\text{var}}, n_{\text{con}}]$$

where $\mathbf{h}_{\text{ct}}$ is a constraint-type histogram, $\mathbf{b}_{\text{dom}}$ is domain-size distribution, $\mathbf{e}_{\tau}$ is step-type one-hot encoding, and normalized problem scale metrics.

Clustering: We apply complete-linkage agglomerative clustering with cosine distance and threshold $\theta = 0.25$ (determined via grid search on development set). Clusters with fewer than 5 members are discarded.

Paradigm Abstraction: For each cluster, we uniformly sample up to 10 representative KDPs, format their solver states and reasoning text, and prompt an LLM to synthesize a reusable paradigm. LLM outputs are parsed into structured (trigger, operation, precondition, postcondition, scope) tuples. Up to 2 retries per cluster are permitted.

Z3 Triple Verification For each candidate paradigm, we execute three independent verification checks:

(1) **Soundness:** Sample 50 random puzzle states satisfying trigger conditions. Test whether paradigm operations maintain Z3 satisfiability:

$$\text{soundness}(P) = \frac{1}{50} \sum_{i=1}^{50} \mathbf{1}[\text{Z3-SAT}(C_i \cup \{\text{op}(P)\})] \geq 0.90$$

(2) **Effect:** Verify that the paradigm operation is genuinely informative — not vacuous and not a tautology under the pre-condition. Two sub-checks must both pass:

- **Non-contradiction:** $\text{Z3-SAT}(\phi_{\text{pre}} \wedge \text{op}(P))$ — the operation is consistent with the pre-condition.
- **Non-vacuousness:** $\text{Z3-SAT}(\phi_{\text{pre}} \wedge \neg \text{op}(P))$ — there exists at least one state where the pre-condition holds but the operation does not. Equivalently, ϕ_{pre} does not entail $\text{op}(P)$. Were this check to fail (i.e. $\phi_{\text{pre}} \wedge \neg \text{op}(P)$ were UNSAT), the paradigm would be entailed by its own trigger and contribute no information.

(3) **Trigger Precision:** Estimate trigger selectivity against a representative constraint pool. Each pool entry is annotated with one or more canonical constraint kinds drawn from a fixed taxonomy $\mathcal{K} = \{\text{position_arithmetic}, \text{position_equality}, \text{position_inequality}, \text{attribute_binding}, \text{distinct}, \text{quantified_exclusion}, \text{integer_bound}, \text{integer_equality}, \text{integer_inequality}, \text{integer_arithmetic}\}$. The trigger of paradigm P fires on sample s iff its declared constraint kinds $\mathcal{K}_P$ have non-empty intersection with the kinds present in s — a structural set-intersection match rather than substring containment, which prevents spurious lexical co-occurrence (e.g., a `position_arithmetic` paradigm does not falsely fire on a generic attribute binding that mentions “position”).

$$\text{precision}(P) = \frac{1}{N_p} \sum_{i=1}^{N_p} \mathbf{1}[\mathcal{K}_P \cap \mathcal{K}_{s_i} = \emptyset] \geq \tau_p$$

Here, $N_p = 50$ random subsets of size 2–5 are drawn from a pool of $|\text{pool}| = 18$ annotated constraints spanning ZebraLogic-domain and generic arithmetic patterns; τ_p is the non-firing rate floor.

Paradigms passing all three checks are persisted in Paradigm Library $\mathcal{L}$. Across 600 training puzzles, offline distillation yields approximately 35 verified paradigms (40:1 compression ratio).

Online Stage: Paradigm-Guided Inference

Two-Layer Paradigm Retrieval For puzzle $\mathcal{P}^*$, at each step t , we extract constraint-type signature $\sigma(S_t)$ and apply:

Layer-1 (Structured Type Matching): Return paradigms $P \in \mathcal{L}$ ranked by a weighted combination of scope-Jaccard similarity with $\sigma(S_t)$ and confidence. Crucially, type-set inclusion alone is too weak: two puzzles with the same set of constraint types may differ substantially in multiplicity (e.g. one ordering constraint vs. four). To address this, each paradigm’s trigger may carry optional *relational predicates* from a small vocabulary — `count_atleast( $t, n$ )`: at least n constraints of type t must be present; `count_atmost( $t, n$ )`; `cooccur( $t_1, t_2$ )`: both types must be present. Predicates are evaluated against the actual type bag (a list with repetitions) and paradigms whose predicates fail are pruned *before* ranking. This lifts Layer-1 from pure set-membership to a set-with-multiplicity match while remaining LLM-free and $O(|\mathcal{L}|)$. The combined filter typically reduces candidates from 35 to < 5 .

Layer-2 (Semantic Filtering, cost-aware): Layer-2 invokes an additional LLM call to resolve ambiguity among Layer-1 candidates. Because this is the most expensive per-step component, we gate it by an activation policy. Under our default `complexity_gated` policy, Layer-2 runs only when (i) Layer-1 returns ≥ 2 candidates *and* (ii) either the puzzle size meets a complexity floor (default 25 constraints, $\approx 5 \times 5$ ZebraLogic) or at least one UNSAT has been observed in the current puzzle. Alternative policies always (legacy behaviour) and `stagnation_only` (Layer-2 deferred until the repair memory flags stagnation) are also supported. When Layer-2 is gated off, the Layer-1 top-1 candidate is retained — guidance is never fully suppressed, and the downstream Z3 consistency pre-check still filters obviously incompatible hints.

Consistency Pre-Check and Hint Injection Before paradigm injection, execute Z3 consistency verification:

$$\text{consistent}(P, S_t) = \text{Z3-SAT}(C_t \cup \{\text{op}(P)\})$$

Only paradigms returning SAT are injected as “soft” (advisory) hints, not hard constraints. This prevents LLM misapplication when paradigms don’t fully align with current state.

Real-Time Z3 Verification and UNSAT Attribution After LLM generates inference action a_t , add to solver via constraint registry (enabling tracking) and call Z3:

SAT Case: Update domain assignments, save as potential revert checkpoint. Continue solving.

UNSAT Case: Extract UNSAT core U_t and perform two-level causal attribution.

Primary attribution (coarse):

$$\text{attr}(a_t, U_t) = \begin{cases} \text{NEW_ASSERTION} & \text{if } a_t \in U_t \\ \text{LEGACY_ERROR} & \text{if } a_t \notin U_t \end{cases}$$

If `LEGACY_ERROR`, the contradiction predates a_t ; revert a_t and diagnose the pre-existing set. If `NEW_ASSERTION`, a_t itself is responsible.

Secondary classification (fine-grained) refines `NEW_ASSERTION` into a structured *ErrorType*. Classification is performed by an AST pass over the parsed Z3 expression rather than by substring matching on the raw constraint string — this makes the classifier robust to Z3’s many equivalent surface forms (e.g. `Not(x > 0)` is recognised as a flipped form of `x <= 0`).

- **SyntaxError** — the assertion fails Z3 parsing.
- **ScopeError** — the parsed expression references an identifier outside the currently-declared variable set.
- **SemanticFlip** — the AST contains contradictory comparison operators on identical operand pairs (e.g. $x \geq k \wedge x \leq k-1$, or $x = k \wedge \neg(x = k)$). Detection traverses the expression with a polarity-tracking walk so that negations under `Not` are normalised before atom comparison.
- **OverConstraint** — default: the new assertion legally tightens the formula beyond satisfiability.

The *ErrorType* label is stored in R_t and used by the strategy escalation to select the most appropriate repair class (e.g., `SemanticFlip` triggers constraint-direction repair rather than bound relaxation).

Repair Trajectory Memory

Memory Data Structure Repair Trajectory Memory $\mathcal{M}$ maintains ordered records $\{R_1, \dots, R_L\}$, each a six-tuple:

$$R_t = (e_t, U_t, \hat{U}_t, \alpha_t, o_t, U'_t)$$

where:

- e_t is error type classification
- U_t is UNSAT core (constraint IDs)
- $\hat{U}_t = \text{SHA256}(\text{sorted}(U_t))$ is core fingerprint for rapid comparison
- α_t is repair action (type, target constraint, summary text, embedding)
- o_t is repair outcome (SAT or UNSAT)
- U'_t is new UNSAT core post-repair (if UNSAT)

Repair action embeddings are computed via all-MiniLM-L6-v2 sentence transformer (384-dim), enabling semantic similarity queries.

Stagnation Detection Stagnation occurs when recent repair records show high UNSAT core overlap, indicating that repairs are not addressing the root contradiction:

$$\text{stagnate}(\mathcal{M}, k=3) = 1 \left[\max_{i \neq j \in \text{last-}k} J(\tilde{U}_i, \tilde{U}_j) \geq \tau_s \right]$$

where Jaccard similarity $J(A, B) = |A \cap B| / |A \cup B|$ is computed over *canonicalised* cores $\tilde{U}_t$ (see below) drawn from the last $k = 3$ records. We use the *maximum* (any-pair) rather than the minimum (all-pairs) as the trigger criterion: early-warning detection is preferable to late-stage detection because the former preserves more of the repair budget for recovery actions.

Z3 core non-uniqueness and canonicalisation. Z3’s `unsat_core()` returns *one* minimal unsatisfiable subset, not the unique one — different solver invocations on the same logically unsatisfiable constraint set may return different cores depending on assumption ordering and internal heuristics. Naively comparing raw cores can therefore both *under-detect* stagnation (the same underlying contradiction surfaces as different core sets across iterations) and *over-detect* it (two unrelated contradictions share a common high-frequency constraint such as `Distinct(...)`). To mitigate this we canonicalise each core via (i) constraint-string normalisation (whitespace, operator aliasing such as `Not(x>0) ↔ x<=0`), and (ii) variable renaming to a canonical schema (`v1, v2, ...`) by first-occurrence order. The fingerprint $\hat{U}_t$ stored in each repair record is the SHA-256 of the canonicalised core, making cross-iteration comparison robust to syntactically-equivalent but lexically-different core listings.

Loop Detection Loop detection identifies when a proposed repair duplicates a historical attempt. We combine two signals so that LLM paraphrase noise does not defeat detection:

(i) **Structured signal (primary).** Each repair action carries a structured triple (τ, c, π) where τ is the repair type (drawn from a closed vocabulary of eight classes such as `relax_bound`, `split_constraint`), c is the target constraint id, and π is a parameter signature (e.g. `+1, *2, removed_clause_3`) summarising the atomic edit. A loop is flagged when this triple exactly matches that of any prior record, regardless of natural-language phrasing.

(ii) **Semantic signal (fallback).** When the structured triple is absent or does not match, we fall back to cosine similarity between sentence-transformer embeddings of the action summaries:

$$\text{loop}(\alpha_{\text{new}}) = 1 \left[((\tau, c, \pi)_{\text{new}} = (\tau, c, \pi)_t \text{ for some } t) \vee \left(\max_t \cos(\mathbf{e}_{\text{new}}, \mathbf{e}_t) \geq \tau_\ell \right) \right]$$

with $\tau_\ell = 0.90$. The structured-signal-first design lets PRISM detect loops even when the LLM paraphrases (which routinely produces cosines in the 0.78-0.85 range — below the safe semantic threshold).

Four-Level Strategy Escalation Upon stagnation or loop detection:

1. **L1 - Switch Target:** Preserve repair type but target different constraint from UNSAT core
2. **L2 - Switch Type:** Change repair strategy class (e.g., `relax-bound` → `split-constraint`)
3. **L3 - Revert Checkpoint (multi-level stack):** Pop the most recent SAT-adjacent state from a bounded checkpoint stack (depth ≤ 5) and roll back the solver to it; resume from an alternative inference path. Successive L3 escalations consume successive entries from the stack, enabling the system to roll back further when a single-step revert does not break stagnation. When the stack is exhausted, the next escalation falls through to L4.
4. **L4 - Full Retranslation:** Discard current formalization; re-translate all constraints from natural language with error history as guidance

Termination property. The online phase is guaranteed to terminate within a bounded number of LLM invocations per puzzle. Concretely: (i) the outer repair loop is capped by `max_repair_rounds` ($R=5$ in our default configuration); (ii) L4 (Full Retranslation) is triggered at most once per puzzle — a second attempt at L4 is suppressed and the puzzle is marked unsolved; (iii) L3 (Revert Checkpoint) pops at most $|\text{checkpoint-stack}|$ entries, and an exhausted stack falls through to L4. Thus the worst-case per-puzzle LLM-call budget is $1(\text{translate}) + R(\text{repair rounds}) + 1(\text{L4 retranslate}) = R + 2$. A short proof sketch is given in Appendix B.

Paradigm Write-Back and Library Maintenance

When a repair (or a fresh inference step) succeeds, PRISM does *not* immediately admit the corresponding pattern into the live paradigm library $\mathcal{L}$. We adopt a *staged write-back* policy:

1. **Candidate staging (online, $O(1)$).** On a successful repair / SAT continuation, the responsible (trigger features, operation, inferred pre/post-condition) tuple is appended to a candidate pool $\mathcal{L}^\dagger$. No verification is performed online, so the inference loop remains low-latency.
2. **Batched re-verification (offline / periodic).** Every K solved puzzles (or end-of-evaluation), candidates in $\mathcal{L}^\dagger$ are de-duplicated against $\mathcal{L}$ (by trigger-kind signature and operation-template hash) and re-run through the same triple verification protocol (Section 3.3.4) used in offline distillation.
3. **Promotion criterion.** Candidates that pass all three checks (soundness $\geq \tau_s$, effect, precision $\geq \tau_p$) are admitted to $\mathcal{L}$ with their support count and confidence initialised from the screening trials; failed candidates are discarded with provenance retained for later analysis.
4. **Existing-paradigm updates.** When an online step matches an existing $P \in \mathcal{L}$ and succeeds, only its $\langle \text{support_count}, \text{confidence} \rangle$ counters are updated atomically — no schema change, no re-verification.

This staged policy keeps the online critical path free of expensive verification while ensuring that $\mathcal{L}$ retains the same SMT-screened quality guarantee at all times. The candidate pool $\mathcal{L}^\dagger$ is not used for retrieval during the online phase.

Computational Cost Analysis

PRISM’s cost has two components that should be reported separately because they amortise differently across deployments.

Offline (one-time, amortised over the test set). The offline phase issues LLM calls in three places: (i) trajectory collection, $N_{\text{train}} \cdot r \cdot \bar{T}$ where $r=3$ is sampling repetition and $\bar{T}$ is the mean trajectory length; (ii) paradigm abstraction, at most $2|\mathcal{C}|$ calls for $|\mathcal{C}|$ clusters with up to two retries each; (iii) batched re-verification of online write-back candidates, $\propto |\mathcal{L}^\dagger|$. Z3 work scales as $50 \cdot |\mathcal{P}_{\text{candidate}}|$ verification trials. The entire offline pipeline runs once and its cost is amortised over every subsequent test puzzle: $C_{\text{off,amortised}} = C_{\text{offline,total}} / N_{\text{test}}$.

Online (per puzzle, worst case). Per the termination property (Section 3.5), the online phase invokes the LLM at most $R + 2$ times per puzzle: one translation, at most R repair-round inferences, and at most one L4 retranslation. The Layer-2 semantic match adds an extra LLM call per step *only* when the cost-aware policy fires (Section 3.4.1); in the default `complexity-gated` policy this contributes $\leq R$ additional calls and is suppressed entirely on easy instances. Z3 work per step is bounded by one consistency pre-check + one solver call + one UNSAT-core extraction.

Amortised total per test puzzle.

$$C_{\text{total}}(p) = \underbrace{C_{\text{off,amortised}}}_{\text{shared offline cost}} + \underbrace{C_{\text{online}}(p)}_{\leq (R+2) + \#\text{[L2 fires]} \cdot R \text{ LLM calls}}.$$

For deployments where N_{test} is large (typical), $C_{\text{off,amortised}}$ becomes negligible and the dominant cost is the bounded online budget. We report both components separately in Section 4 so that the comparison with neural-symbolic baselines reflects the realised, not headline, cost.

Experiments

Experimental Setup

Benchmark: ZebraLogic (Lin et al., 2025) contains logical grid puzzles ranging from 3×5 to 6×6 (30 to 180 variables, 15-80 constraints). We evaluate on three representative sizes: 4×4 (easy), 5×5 (medium), 6×6 (hard).

Baselines:

1. Chain-of-Thought (CoT) prompting
2. Logic-LM (Pan et al., 2023)
3. ExpeL with CSP adaptation
4. Neural-Symbolic baseline (LLM+Z3, no memory)
5. Verified Few-Shot (direct paradigm injection without library)
6. PRISM ablations: (a) memory only, (b) paradigm library only

Metrics: Solving accuracy, average LLM calls per puzzle, average repair rounds, paradigm trigger rate, paradigm hit rate.

Results

[This section will contain: (1) Accuracy improvements, (2) Cost analysis (LLM calls), (3) Repair efficiency (stagnation reduction), (4) Paradigm transfer analysis, (5) Ablation studies]

Limitations and Failure Modes

We discuss known failure modes that the screening and escalation mechanisms only partially mitigate, so the reader can calibrate the strength of our claims.

(1) Probabilistic, not formal, paradigm soundness. Triple verification samples $N_p = 50$ constraint subsets and accepts paradigms whose empirical soundness exceeds $\tau_s = 0.90$. Up to 10% of admitted paradigms may therefore fail on a held-out solver state. The online consistency pre-check filters most such failures before injection, but residual

mis-application remains possible on novel constraint configurations not represented in the offline pool.

(2) **Residual UNSAT-core non-uniqueness.** Canonicalisation (operator aliasing + first-occurrence variable renaming) substantially reduces the divergence of Z3’s chosen cores across iterations, but does not eliminate it: cases where two structurally different minimal cores correspond to the same underlying contradiction can still slip past Jaccard-based stagnation detection. Stronger mitigations such as MUS enumeration are left to future work due to their cost.

(3) **L4 retranslation can degrade good translations.** The L4 escalation discards the current formalisation and asks the LLM to retranslate from natural language. In rare cases — particularly when the latent contradiction was due to a downstream over-constraint rather than a translation error — L4 will produce a worse translation than the one it replaced. We mitigate this by allowing L4 at most once per puzzle, but the recovery action remains coarse.

(4) **LLM-induced paradigm idiosyncrasies.** The offline distillation invokes an LLM to synthesise paradigm bodies from cluster representatives. Different LLM choices (vendor, model, temperature) can produce libraries with markedly different coverage and trigger style; the triple verification protocol bounds quality but does not eliminate this dependency. Cross-LLM paradigm portability is an open question.

(5) **Higher-order logical constructs.** The current Z3 wrapper supports quantified constraints via `ForAll` / `Exists`, but more advanced theory combinations (uninterpreted functions over richer sorts, recursive datatypes) are out of scope. Extending PRISM to those theories would require lifting the canonicalisation and AST-based `ErrorType` classification beyond the current arithmetic + propositional fragment.

Conclusion

PRISM introduces a memory-augmented neuro-symbolic framework tailored to CSP solving. By using Z3 as an automated quality-screening oracle for candidate paradigms, encoding paradigms with SMT pre/post-conditions, and implementing adaptive repair strategies via a typed trajectory memory, PRISM achieves consistent improvements over neural-symbolic and memory-based baselines.

Key findings: (1) experience accumulation yields disproportionately larger gains on harder problems, (2) SMT-based empirical screening of paradigms enables more reliable knowledge transfer than unverified natural-language heuristics, (3) adaptive strategy escalation effectively breaks repair stagnation, (4) paradigms transfer across problem scales and, to a more limited extent, across problem types.

Future work includes: extending to other symbolic domains beyond CSPs, investigating curriculum learning for paradigm discovery, and exploring interaction with classical CSP solvers.

Acknowledgments

We acknowledge support from [funding sources]. We thank [individuals] for valuable discussions.

References